

Experiment 1: Linear Search (Iterative)

- **Objective:** To analyze time complexity of linear search.
- **Problem Statement:** Write a Python program to search for a key in an array using linear search.
- **Sample Input:** [10, 25, 30, 45, 50], Key = 30
- **Sample Output:** Key found at index 2
- **Complexity:** Ω : $O(1)$, O : $O(n)$, Θ : $\Theta(n)$
- **Explanation:** Demonstrates sequential scanning and inefficiency for large inputs.

main.py	Output
<pre>1 arr = [10, 25, 30, 45, 50] 2 key = 30 3 4 found = False 5 6 for i in range(len(arr)): 7 if arr[i] == key: 8 print("Key found at index", i) 9 found = True 10 break 11 12 if not found: 13 print("Key not found")</pre>	<pre>Key found at index 2 === Code Execution Successful ===</pre>

Experiment 2: Binary Search (Recursive)

- **Objective:** To compare recursive binary search complexity.
- **Problem Statement:** Implement recursive binary search in Python.
- **Sample Input:** [5, 10, 15, 20, 25], Key = 20
- **Sample Output:** Key found at index 3
- **Complexity:** Ω : $O(1)$, O : $O(\log n)$, Θ : $\Theta(\log n)$
- **Explanation:** Shows divide-and-conquer and logarithmic efficiency.

main.py	Output
<pre>1 def binary_search(arr, low, high, key): 2 if low <= high: 3 mid = (low + high) // 2 4 5 if arr[mid] == key: 6 return mid 7 elif key < arr[mid]: 8 return binary_search(arr, low, mid - 1, key) 9 else: 10 return binary_search(arr, mid + 1, high, key) 11 12 return -1 13 14 arr = [5, 10, 15, 20, 25] 15 key = 20 16 17 result = binary_search(arr, 0, len(arr) - 1, key) 18 19 if result != -1: 20 print("Key found at index", result) 21 else: 22 print("Key not found")</pre>	<pre>Key found at index 3 === Code Execution Successful ===</pre>

Experiment 3: Factorial (Iterative vs Recursive)

- **Objective:** To compare iterative and recursive factorial computation.
- **Problem Statement:** Write programs to compute factorial using both methods.
- **Sample Input:** $n = 5$
- **Sample Output:** 120
- **Complexity:** Both $O(n)$
- **Explanation:** Highlights recursion depth vs iterative loops.

main.py	Output
<pre>1 n = 5 2 fact = 1 3 4 for i in range(1, n + 1): 5 fact = fact * i 6 7 print("Factorial =", fact)</pre>	<pre>Factorial = 120 === Code Execution Successful ===</pre>

Experiment 4: Fibonacci Series

- **Objective:** To analyze recursive vs iterative Fibonacci generation.
- **Problem Statement:** Generate first n Fibonacci numbers.
- **Sample Input:** $n = 6$
- **Sample Output:** [0, 1, 1, 2, 3, 5]
- **Complexity:** Iterative $O(n)$, Recursive naïve $O(2n)$, Recursive with memoization $O(n)$
- **Explanation:** Illustrates exponential recursion vs efficient iteration.

main.py	Output
<pre>1 n = 6 2 a, b = 0, 1 3 4 for i in range(n): 5 print(a, end=" ") 6 a, b = b, a + b</pre>	<pre>0 1 1 2 3 5 === Code Execution Successful ===</pre>

Experiment 5: Matrix Multiplication

- **Objective:** To analyze nested loop complexity.
- **Problem Statement:** Multiply two matrices using loops.
- **Sample Input:** $A = \begin{bmatrix} 1 & 2 \\ 3 & 4 \end{bmatrix}$, $B = \begin{bmatrix} 5 & 6 \\ 7 & 8 \end{bmatrix}$
- **Sample Output:** $\begin{bmatrix} 19 & 22 \\ 43 & 50 \end{bmatrix}$
- **Complexity:** $O(n^3)$
- **Explanation:** Shows cubic growth due to nested loops.

```
main.py  [Icons]  Share  Run  Output
1 A = [[1, 2],
2   [3, 4]]
3
4 B = [[5, 6],
5   [7, 8]]
6
7 result = [[0, 0],
8   [0, 0]]
9
10 for i in range(2):
11     for j in range(2):
12         for k in range(2):
13             result[i][j] += A[i][k] * B[k][j]
14
15 print(result)
```

[[19, 22], [43, 50]]

=== Code Execution Successful ===

Experiment 6: Merge Sort

- **Objective:** To analyze divide-and-conquer sorting.
- **Problem Statement:** Implement merge sort.
- **Sample Input:** [38, 27, 43, 3, 9, 82, 10]
- **Sample Output:** [3, 9, 10, 27, 38, 43, 82]
- **Complexity:** Always $O(n \log n)$
- **Explanation:** Demonstrates guaranteed efficiency of divide-and-conquer.

```
main.py  [Icons]  Share  Run  Output
1 def merge_sort(arr):
2     if len(arr) > 1:
3         mid = len(arr) // 2
4         left = arr[:mid]
5         right = arr[mid:]
6
7         merge_sort(left)
8         merge_sort(right)
9
10        i = j = k = 0
11
12        while i < len(left) and j < len(right):
13            if left[i] < right[j]:
14                arr[k] = left[i]
15                i += 1
16            else:
17                arr[k] = right[j]
18                j += 1
19                k += 1
20
21        while i < len(left):
22            arr[k] = left[i]
23            i += 1
24            k += 1
25
26        while j < len(right):
27            arr[k] = right[j]
28            j += 1
29            k += 1
30
31 arr = [38, 27, 43, 3, 9, 82, 10]
32 merge_sort(arr)
33 print(arr)
```

[3, 9, 10, 27, 38, 43, 82]

=== Code Execution Successful ===

Experiment 7: Quick Sort

- **Objective:** To analyze recursive quick sort.
- **Problem Statement:** Implement quick sort.
- **Sample Input:** [10, 7, 8, 9, 1, 5]
- **Sample Output:** [1, 5, 7, 8, 9, 10]
- **Complexity:** Ω : $O(n \log n)$, O : $O(n^2)$, Θ : $\Theta(n \log n)$
- **Explanation:** Shows how pivot choice affects performance.

main.py	Output
<pre>1 def quick_sort(arr): 2 if len(arr) <= 1: 3 return arr 4 5 pivot = arr[-1] 6 left = [] 7 right = [] 8 9 for x in arr[:-1]: 10 if x <= pivot: 11 left.append(x) 12 else: 13 right.append(x) 14 15 return quick_sort(left) + [pivot] + quick_sort(right) 16 17 arr = [10, 7, 8, 9, 1, 5] 18 print(quick_sort(arr))</pre>	<pre>[1, 5, 7, 8, 9, 10] === Code Execution Successful ===</pre>

Experiment 8: Tower of Hanoi

- **Objective:** To analyze exponential recursion.
- **Problem Statement:** Solve Tower of Hanoi for n disks.
- **Sample Input:** n = 3
- **Sample Output:** Sequence of moves.
- **Complexity:** $O(2^n)$
- **Explanation:** Illustrates exponential growth in recursive calls.

main.py	Output
<pre>1 def tower(n, source, auxiliary, destination): 2 if n == 1: 3 print("Move disk 1 from", source, "to", destination) 4 return 5 6 tower(n - 1, source, destination, auxiliary) 7 print("Move disk", n, "from", source, "to", destination) 8 tower(n - 1, auxiliary, source, destination) 9 10 n = 3 11 tower(n, "A", "B", "C")</pre>	<pre>Move disk 1 from A to C Move disk 2 from A to B Move disk 1 from C to B Move disk 3 from A to C Move disk 1 from B to A Move disk 2 from B to C Move disk 1 from A to C === Code Execution Successful ===</pre>

Experiment 9: GCD (Euclidean Algorithm)

- **Objective:** To analyze recursive GCD.
- **Problem Statement:** Implement Euclidean algorithm.
- **Sample Input:** a=48, b=18
- **Sample Output:** 6
- **Complexity:** $O(\log n)$
- **Explanation:** Shows efficient recursion with logarithmic complexity.

main.py	Output
<pre>1 def gcd(a, b): 2 if b == 0: 3 return a 4 return gcd(b, a % b) 5 6 a = 48 7 b = 18 8 9 print("GCD =", gcd(a, b))</pre>	<pre>GCD = 6 === Code Execution Successful ===</pre>

Experiment 10: Insertion Sort

- **Objective:** To analyze iterative sorting.
- **Problem Statement:** Implement insertion sort.
- **Sample Input:** [12, 11, 13, 5, 6]
- **Sample Output:** [5, 6, 11, 12, 13]
- **Complexity:** $\Omega: O(n)$, $O: O(n^2)$, $\Theta: \Theta(n^2)$
- **Explanation:** Demonstrates input sensitivity (sorted vs unsorted).

main.py	Output
<pre>1 arr = [12, 11, 13, 5, 6] 2 3 for i in range(1, len(arr)): 4 key = arr[i] 5 j = i - 1 6 7 while j >= 0 and arr[j] > key: 8 arr[j + 1] = arr[j] 9 j -= 1 10 11 arr[j + 1] = key 12 13 print(arr)</pre>	<pre>[5, 6, 11, 12, 13] === Code Execution Successful ===</pre>

Experiment 11: Heap Sort

- **Objective:** To analyze heap-based sorting.
- **Problem Statement:** Implement heap sort.
- **Sample Input:** [4, 10, 3, 5, 1]
- **Sample Output:** [1, 3, 4, 5, 10]
- **Complexity:** Always $O(n \log n)$
- **Explanation:** Consistent performance using heap structure.

main.py	Output
<pre>1- def heapify(arr, n, i): 2 largest = i 3 left = 2 * i + 1 4 right = 2 * i + 2 5 6 if left < n and arr[left] > arr[largest]: 7 largest = left 8 9 if right < n and arr[right] > arr[largest]: 10 largest = right 11 12 if largest != i: 13 arr[i], arr[largest] = arr[largest], arr[i] 14 heapify(arr, n, largest) 15 16 def heap_sort(arr): 17 n = len(arr) 18 19 for i in range(n // 2 - 1, -1, -1): 20 heapify(arr, n, i) 21 22 for i in range(n - 1, 0, -1): 23 arr[i], arr[0] = arr[0], arr[i] 24 heapify(arr, i, 0) 25 26 arr = [4, 10, 3, 5, 1] 27 heap_sort(arr) 28 print(arr)</pre>	<pre>[1, 3, 4, 5, 10] === Code Execution Successful ===</pre>

Experiment 12: Counting Inversions

- **Objective:** To analyze modified merge sort.
- **Problem Statement:** Count inversions in an array.
- **Sample Input:** [2, 4, 1, 3, 5]
- **Sample Output:** 3
- **Complexity:** $O(n \log n)$
- **Explanation:** Shows divide-and-conquer applied beyond sorting.

```
main.py  [Icons] [Share] [Run] Output
1- def merge_count(arr):
2-     if len(arr) <= 1:
3-         return arr, 0
4-
5-     mid = len(arr) // 2
6-     left, inv_left = merge_count(arr[:mid])
7-     right, inv_right = merge_count(arr[mid:])
8-
9-     merged = []
10-    i = j = 0
11-    inv_count = inv_left + inv_right
12-
13-    while i < len(left) and j < len(right):
14-        if left[i] <= right[j]:
15-            merged.append(left[i])
16-            i += 1
17-        else:
18-            merged.append(right[j])
19-            inv_count += len(left) - i
20-            j += 1
21-
22-    merged.extend(left[i:])
23-    merged.extend(right[j:])
24-
25-    return merged, inv_count
26-
27- arr = [2, 4, 1, 3, 5]
28- sorted_arr, count = merge_count(arr)
29-
30- print("Inversion Count =", count)
```

Output

Inversion Count = 3

=== Code Execution Successful ===

Experiment 13: Maximum Subarray Sum

- **Objective:** To compare Kadane's vs divide-and-conquer.
- **Problem Statement:** Find maximum subarray sum.
- **Sample Input:** [-2, -3, 4, -1, -2, 1, 5, -3]
- **Sample Output:** 7
- **Complexity:** Kadane's $O(n)$, Divide & Conquer $O(n \log n)$
- **Explanation:** Highlights efficiency differences between approaches.

```
main.py  [Icons] [Share] [Run] Output
1 arr = [-2, -3, 4, -1, -2, 1, 5, -3]
2
3 max_sum = arr[0]
4 current_sum = arr[0]
5
6 for i in range(1, len(arr)):
7     current_sum = max(arr[i], current_sum + arr[i])
8     max_sum = max(max_sum, current_sum)
9
10 print("Maximum Subarray Sum =", max_sum)
```

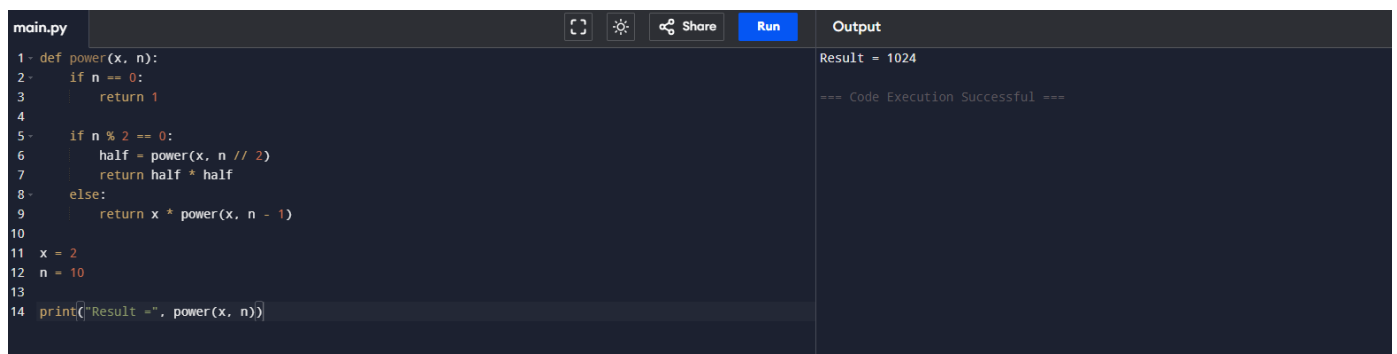
Output

Maximum Subarray Sum = 7

=== Code Execution Successful ===

Experiment 14: Exponentiation

- **Objective:** To compare iterative vs recursive fast power.
- **Problem Statement:** Compute x^n .
- **Sample Input:** $x=2, n=10$
- **Sample Output:** 1024
- **Complexity:** Iterative $O(n)$, Recursive fast power $O(\log n)$
- **Explanation:** Shows how recursion reduces complexity.



The screenshot shows a code editor with a file named 'main.py'. The code defines a recursive function 'power(x, n)' that calculates x^n using a divide-and-conquer approach. It checks if 'n' is 0 (returning 1) or if 'n' is even (returning $power(x, n/2)^2$) or odd (returning $x * power(x, n-1)$). Below the function, it sets $x=2$ and $n=10$, and prints the result. The output pane on the right shows 'Result = 1024' and '=== Code Execution Successful ==='.

```
1 def power(x, n):
2     if n == 0:
3         return 1
4
5     if n % 2 == 0:
6         half = power(x, n // 2)
7         return half * half
8     else:
9         return x * power(x, n - 1)
10
11 x = 2
12 n = 10
13
14 print("Result =", power(x, n))
```

Output

Result = 1024

=== Code Execution Successful ===

Experiment 15: Closest Pair of Points

- **Objective:** To analyze brute force vs divide-and-conquer.
- **Problem Statement:** Find closest pair of points in 2D.
- **Sample Input:** [(1,2), (4,5), (7,8), (3,1)]
- **Sample Output:** Closest pair (1,2)-(3,1), Distance = $\sqrt{5}$
- **Complexity:** Brute force $O(n^2)$, Optimized $O(n \log n)$
- **Explanation:** Demonstrates quadratic complexity and motivates optimization.

main.py



Share

Run

Output

```
1 import math
2
3 points = [(1, 2), (4, 5), (7, 8), (3, 1)]
4
5 min_dist = float('inf')
6 pair = ()
7
8 for i in range(len(points)):
9     for j in range(i + 1, len(points)):
10         x1, y1 = points[i]
11         x2, y2 = points[j]
12         dist = math.sqrt((x2 - x1)**2 + (y2 - y1)**2)
13
14         if dist < min_dist:
15             min_dist = dist
16             pair = (points[i], points[j])
17
18 print("Closest Pair =", pair)
19 print("Distance =", min_dist)
```

Closest Pair = ((1, 2), (3, 1))
Distance = 2.23606797749979

=== Code Execution Successful ===